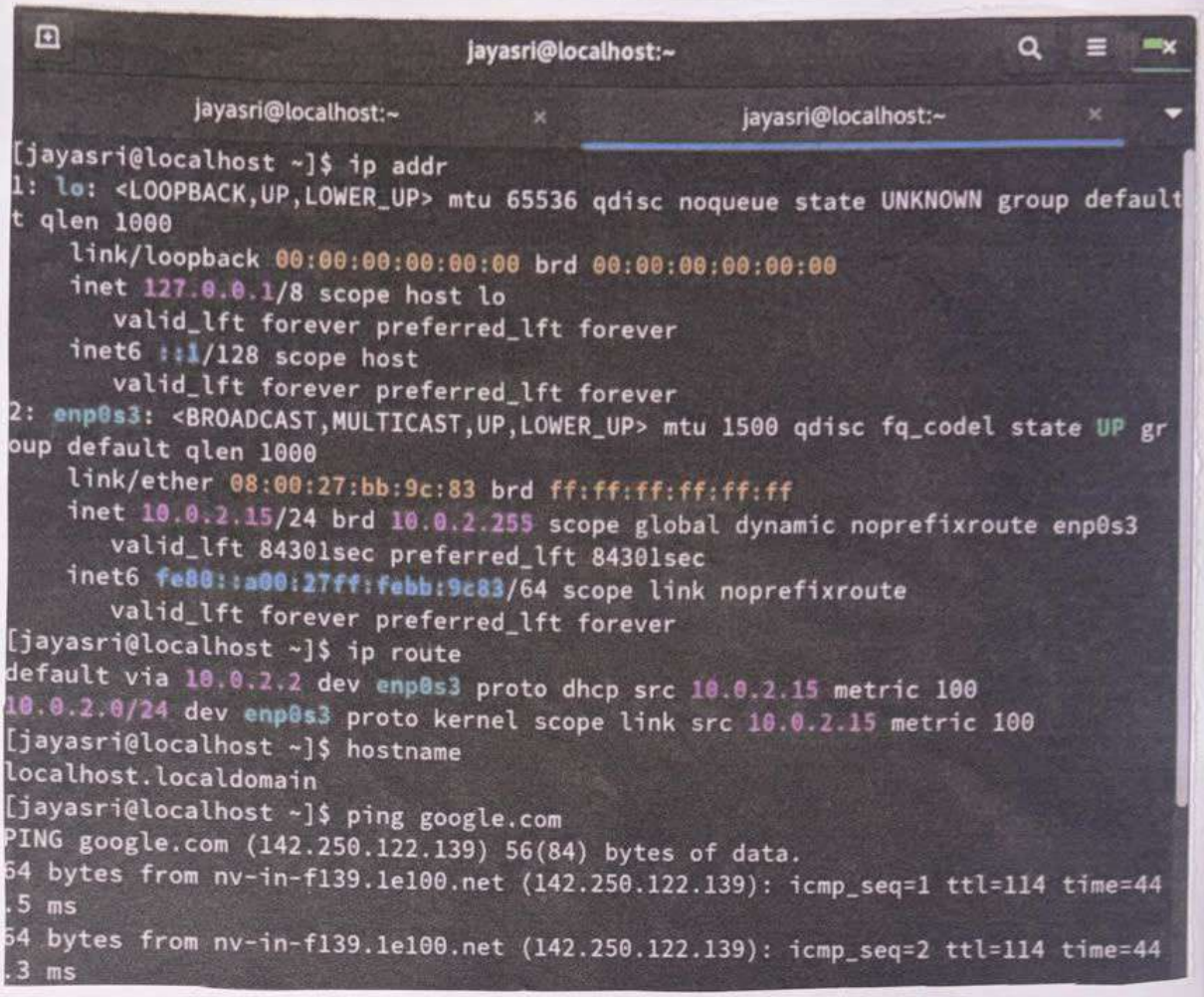
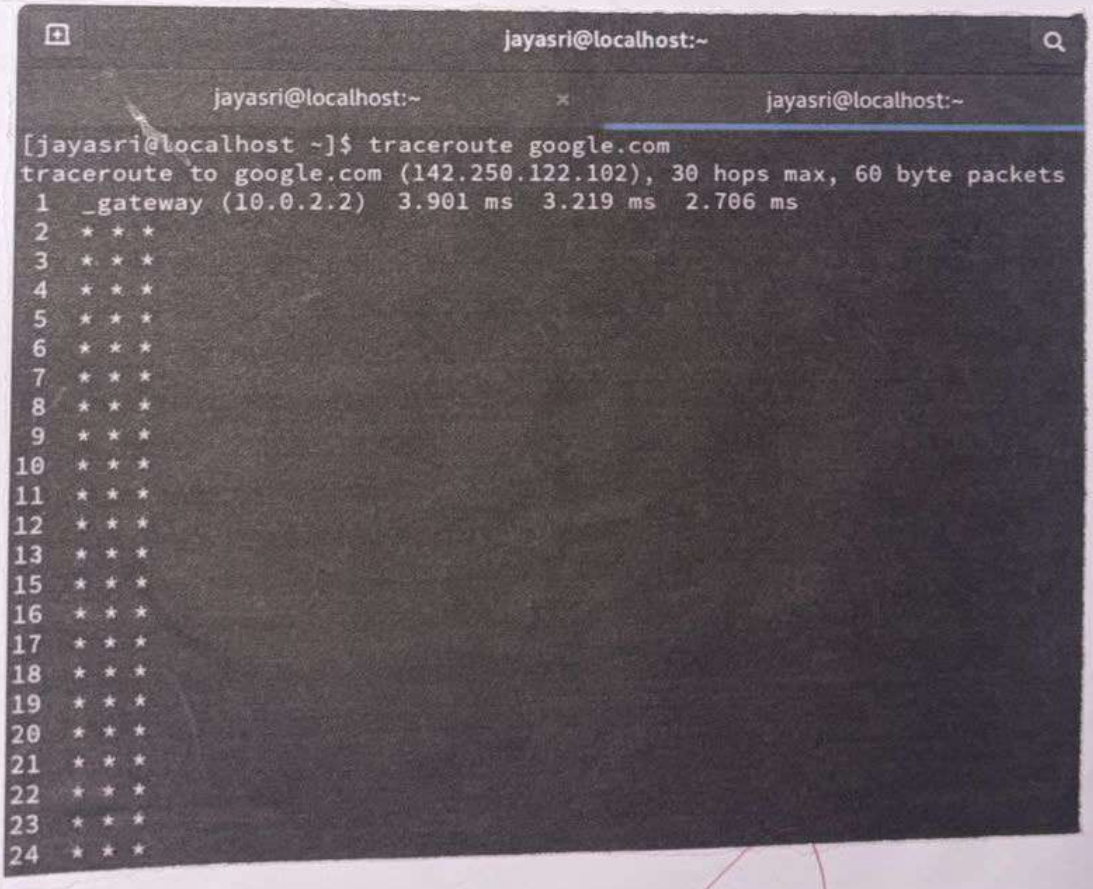

jayasri@localhost:~
jayasri@localhost:~
jayasri@localhost:~
[jayasri@localhost ~]$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default
t qlen 1000
link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
inet 127.0.0.1/8 scope host lo
valid_lft forever preferred_lft forever
inet6 ::1/128 scope host
valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gr
oup default qlen 1000
link/ether 08:00:27:bb:9c:83 brd ff:ff:ff:ff:ff:ff
inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
valid_lft 84301sec preferred_lft 84301sec
inet6 fe80::a00:27ff:febb:9c83/64 scope link noprefixroute
valid_lft forever preferred_lft forever
[jayasri@localhost ~]$ ip route
default via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100
[jayasri@localhost ~]$ hostname
localhost.localdomain
[jayasri@localhost ~]$ ping google.com
PING google.com (142.250.122.139) 56(84) bytes of data.
64 bytes from nv-in-f139.1e100.net (142.250.122.139): icmp_seq=1 ttl=114 time=44
.5 ms
64 bytes from nv-in-f139.1e100.net (142.250.122.139): icmp_seq=2 ttl=114 time=44
.3 ms


jayasri@localhost:~
jayasri@localhost:~
jayasri@localhost:~
[jayasri@localhost ~]$ traceroute google.com
traceroute to google.com (142.250.122.102), 30 hops max, 60 byte packets
1 _gateway (10.0.2.2) 3.901 ms 3.219 ms 2.706 ms
2 * * *
3 * * *
4 * * *
5 * * *
6 * * *
7 * * *
8 * * *
9 * * *
10 * * *
11 * * *
12 * * *
13 * * *
15 * * *
16 * * *
17 * * *
18 * * *
19 * * *
20 * * *
21 * * *
22 * * *
23 * * *
24 * * *

Exp no. 1
22/07/26

BASIC NETWORKING COMMANDS IN LINUX AND WINDOWS OPERATING SYSTEM

AIM:

To study and execute basic networking commands in windows and Linux operating system for viewing network configuration, testing connectivity, checking routine information, monitoring network status, and trouble shooting.

CODE:

Open Commands prompts

Commands:

```
ipconfig
ipconfig /all
Ping google.com
tracert google.com
nslookup google.com
route print
netstat
netstat -a
get mac
hostname
arp - a
```

Open Linux Networking Commands:

```
ip addr
if config
ip route
hostname
ping google.com
traceroute google.com
Sudo apt install traceroute
nslookup google.com
```

```
[jayasri@localhost ~]$ nslookup google.com
Server:         192.168.1.1
Address:        192.168.1.1#53

Non-authoritative answer:
Name:   google.com
Address: 142.250.122.139
Name:   google.com
Address: 142.250.122.113
```

```
[jayasri@localhost ~]$ route
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
default         _gateway        0.0.0.0         UG    100    0        0 enp0s3
10.0.2.0        0.0.0.0         255.255.255.0   U     100    0        0 enp0s3
[jayasri@localhost ~]$ netstat
Active Internet connections (w/o servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 localhost.localdo:38604 mirrors.ustc.edu.c:http TIME_WAIT
tcp        0      0 localhost.localdo:54522 101.6.15.130:http       TIME_WAIT
tcp        0      0 localhost.localdo:38620 mirrors.ustc.edu.c:http TIME_WAIT
tcp        0      0 localhost.localdo:49892 101.6.15.130:http       TIME_WAIT
tcp        0      0 localhost.localdo:54534 101.6.15.130:http       TIME_WAIT
tcp        0      0 localhost.localdo:49904 101.6.15.130:http       TIME_WAIT
tcp        0      0 localhost.localdo:38510 101.6.15.130:http       TIME_WAIT
tcp        0      0 localhost.localdo:54512 101.6.15.130:http       TIME_WAIT
tcp        0      0 localhost.localdo:38608 mirrors.ustc.edu.c:http TIME_WAIT
tcp        0      0 localhost.localdo:38630 mirrors.ustc.edu.c:http TIME_WAIT
tcp        0      0 localhost.localdo:49918 101.6.15.130:http       TIME_WAIT
udp        0      0 localhost.locald:bootpc _gateway:bootps         ESTABLISHED
Active UNIX domain sockets (w/o servers)
Proto RefCnt Flags       Type       State         I-Node   Path
unix  2      [ ]         DGRAM      CONNECTED     22818    /run/chrony/chronyd.s
ock
unix  2      [ ]         DGRAM                    28072    /run/user/1000/system
d/notify
```

```
[jayasri@localhost ~]$ getmac
bash: getmac: command not found...
[jayasri@localhost ~]$ arp -a
_gateway (10.0.2.2) at 52:54:00:12:35:02 [ether] on enp0s3
[jayasri@localhost ~]$ ip neigh
10.0.2.2 dev enp0s3 lladdr 52:54:00:12:35:02 STALE
[jayasri@localhost ~]$ ip link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT
 group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mo
de DEFAULT group default qlen 1000
    link/ether 08:00:27:bb:9c:83 brd ff:ff:ff:ff:ff:ff
[jayasri@localhost ~]$
```

```
[jayasri@localhost ~]$ ip neigh
10.0.2.2 dev enp0s3 lladdr 52:54:00:12:35:02 STALE
[jayasri@localhost ~]$ ip link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT
 group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP
de DEFAULT group default qlen 1000
    link/ether 08:00:27:bb:9c:83 brd ff:ff:ff:ff:ff:ff
[jayasri@localhost ~]$ cat /etc/resolv.conf
# Generated by NetworkManager
nameserver 192.168.1.1
[jayasri@localhost ~]$ ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT
 group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP
de DEFAULT group default qlen 1000
    link/ether 08:00:27:bb:9c:83 brd ff:ff:ff:ff:ff:ff
[jayasri@localhost ~]$ ss -tuin
Netid  State   Recv-Q   Send-Q        Local Address:Port      Peer Address:Port
udp    ESTAB   0        0          10.0.2.15%enp0s3:68           10.0.2.2:67
[jayasri@localhost ~]$ curl google.com
<HTML><HEAD><meta http-equiv="content-type" content="text/html;charset=utf-8">
<TITLE>301 Moved</TITLE></HEAD><BODY>
<H1>301 Moved</H1>
```

```
net stat - a
arp - a
ip link
cat /etc/ resource.conf
ip link show
ss - tucn
curl google.com

Sudo nmdi con mod "Wired connection 1"
        ipv4. getway 192.168.1.1
Sudo nmdi con mod "Wired Connection 1"
        ipv4. method manual
Sudo nmdi con mod "Wired Connection 1"
        ipv4. dns "8.8.8.8"
Sudo nmdi con up "Wired Connection 1"
```

RESULT:

Static IP address were successfully assigned to computers. Connectivity was verified using ping commands between hosts on the same subnet.

Exp 2 (Study)

LEARNING AND ASSIGNMENT OF IP ADDRESS MANUALLY TO COMPUTERS

AIM:
To understand IP addressing scheme and manually assign static IP addresses to computers on a network.

THEORY:
An IP (Internet protocol) address is a unique numerical label assigned to each device connected to a computer network.

IPV4 addresses are 32-bit numbers written in dotted-decimal notation.

Manual (static) IP assignment is used for servers and network devices requiring a permanent address. DHcp assigns IPs dynamically.

RESULT:
Static IP address were successfully assigned to computers. Connectivity was verified using ping commands between hosts on the same subnet.

exp:3

STUDY OF NETWORK CABLES AND CRIMPING WITH RJ45 CONNECTOR

AIM:

To study different types of network cables and to prepare a straight through and cross over ethernet cable by crimping RJ45 connectors.

THEORY:

1. Twisted pair cable:
   VTP (unshielded) STP (shielded) used in LANS.

Categories:
- → Cat5
- → Cat5e
- → Cat6
- → Cat6a

2. Coaxial cable:
   used in cable TV and other ethernet (10Base2)

10Base5)

Higher shielding.

3. Fiber optical cable:
   Transmitts data as light pulses single mode (long distance) and Multimode (short distance).

Very high bandwidth

→ RJ45 is the standard connector for ethernet cables.

Two wiring standard exists.

RESULT:

Both straight-through and cross-over Ethernet cables were successfully crimped and verified using a cable tester. All 8 pins showed proper continuity.

Exp: 4

```
--------------------------------
Version: 4
Header Length: 20
TTL: 4
Protocol: 17
Source: 172.16.10.251
Destination: 239.255.255.250
Packet Length: 411
--------------------------------
Version: 4
Header Length: 20
TTL: 1
Protocol: 17
Source: 172.16.11.142
Destination: 239.255.255.250
Packet Length: 684
--------------------------------
Version: 4
Header Length: 20
TTL: 1
Protocol: 17
Source: 172.16.10.175
Destination: 224.0.0.252
Packet Length: 56
Stopped

C:\Users\HDC0222020\Downloads>
```

Exp: 4

PACKET SNIFFING USING RAW SOCKETS IN PYTHON

AIM:

To implement a packet sniffer using raw sockets in Python that captures live network traffic and parses Ethernet, IP, and transport-layer (TCP/UDP/ICMP) headers.

THEORY:

Raw sockets: Normal sockets (SOCK_STREAM, SOCK_DGRAM) let an application send/receive data through the OS's protocol. A raw socket (SOCK_RAW) - it gives the program direct access to packets at the IP or even the ethernet.

Why root/admin privileges are required: Raw sockets can read/inject arbitary traffic on the network, which is a security-sensitive capability. The OS restricts their creation to privileged processes.

Step 1: Set up the Lab Environment

1. Use a Linux machine or VM (Ubuntu recommended) - VirtualBox/VMware VM is fine, or WSL2 on Windows.
2. Confirm Python 3 is installed:
python3 --version.
3. Ensure you have root access, since raw sockets require it: sudo -v

Step 2: Understand the Packet Structure You'll Parse

When sniffing at the Ethernet layer on Linux, each captured frame looks like:

[Ethernet Header (14 bytes)]
→ [IP Header (20 bytes, variable with options)]
→ [TCP Header (20+ bytes)/UDP Header (8 bytes)]

Step 3: Create the Raw Socket

Create a file sniffer.py and start with the socket creation:

```
import socket
def create_raw_socket():
 try:
  #AF_PACKET + SOCK_RAW = capture at ethernet (Layer 2) level on
  Linux
      s= socket.socket(( socket.AF_PACKET, SOCKET.SOCK_RAW)
      return s
      except socket.error as e:
          printf"Socket could not be created. Error: {e}")
          exit (1)
```

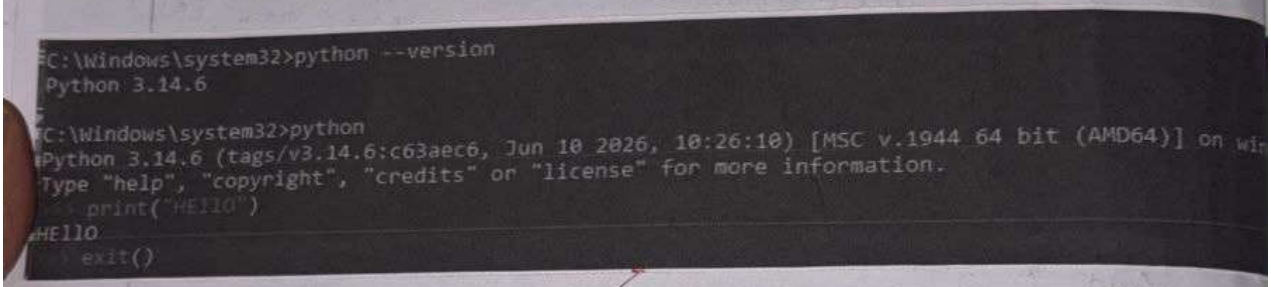
Step 4: Parse the Ethernet Header

Add a function to unpack the first 14 bytes of every captured frame:

```
import struct
def parse_ethernet_header(data):
  eth_header = struct.unpack('!6s6sH', data[:14])
  eth_protocol = socket.ntohs (eth_header[2])
```

Step 5: Parse the IP Header

```
def parse_ip_header (data):
  ip_header = struct.unpack('!BBHHH BBH4s4s', data[:20])
  version_ihl = ip_header[0]
  version = version_ihl >>4
  ttl = ip_header[5]
  protocol = ip_header[6]
  dest_ip = socket.inet_ntoa (ip_header[9])
```


C:\Windows\system32>python --version
Python 3.14.6
C:\Windows\system32>python
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on wi
Type "help", "copyright", "credits" or "license" for more information.
>>> print('HELLO')
HELLO
>>> exit()

Step 7: Write the Main Sniffing Loop

Tie everything together in main():

```
def main():
    s = create_raw_socket()
    print("Sniffer started... Press Ctrl+C to stop.\n")

try:
    while True:
        raw_data, addr = s.recvfrom(65535)

        print("="*60)
        print(f"Ethernet Frame: {src_mac} -> {dest_mac}")
        if protocol == 6: #TCP
        src_port, dest_port, seq, ack, flags)
        elif protocol == 17: #UDP
            icmp_type, icmp_code, checksum = parse_icmp_header

except KeyboardInterrupt:
    print("\nSniffer stopped by user.")
    s.close()

if __name__ == "__main__":
    main()
```

Step 8: Lab Report - What to document

10. Objectives and theory (raw sockets, OSI layers involved).
11. Screenshots of the sniffers running and capturing live traffic.
12. Code with comments (use the sections above)
13. Ethical / legal note - state that packet sniffing was performed.

RESULT:

Packet sniffing using raw sockets in python is completed and verified successfully.

Exp 5

DEVELOP A CUSTOMIZED PING COMMAND TO TEST THE SERVER CONNECTIVITY

AIM:

To develop a customized Ping command in Python that checks the connectivity between a client and a server by sending ICMP Echo Request packets and displaying the response time and status.

REQUIREMENT:

Hardware

- Computer/Laptop
- Internet Connection

Software

- Windows 10/11 or Ubuntu Linux
- Python 3.10 or above

THEORY:

A ping command is a network diagnostic tool used to determine whether a host (server) is reachable over a network. It works using the ICMP (Internet Control Message Protocol)

The customized ping program allows the user to:

- Enter any hostname or IP address.
- Send multiple ping requests
- Display response time
- Show packet loss.
- Indicate whether the server is reachable.

RESULT:

Thus to develop a customized ping command to test the connectivity is completed and verified successfully.

Exp. 6

BULIDING AN ANONYMOUS FTD SCANNAR USING PYTHON

AIM:

To develop a python based anonymous FTD scanner using the built-in ftplib module and determine whether an FTP server permits anonymous perimts.

REQUIREMENTS:

* Windows/Linux computer
* Python 3.x
* Python build-in ftplib module
* An FTP server that you are authorized to test
* Internet / Network connection for remove OTP.
* No separate installation of ftplib is required.

THEORY:

FTP (File Transfer Protocol) is used to transfer file between computers. Python's Ftplib modules provides function to connect to FTP servers, login, list files and close connection. The Scanner attempts to login user g the username anonymous.

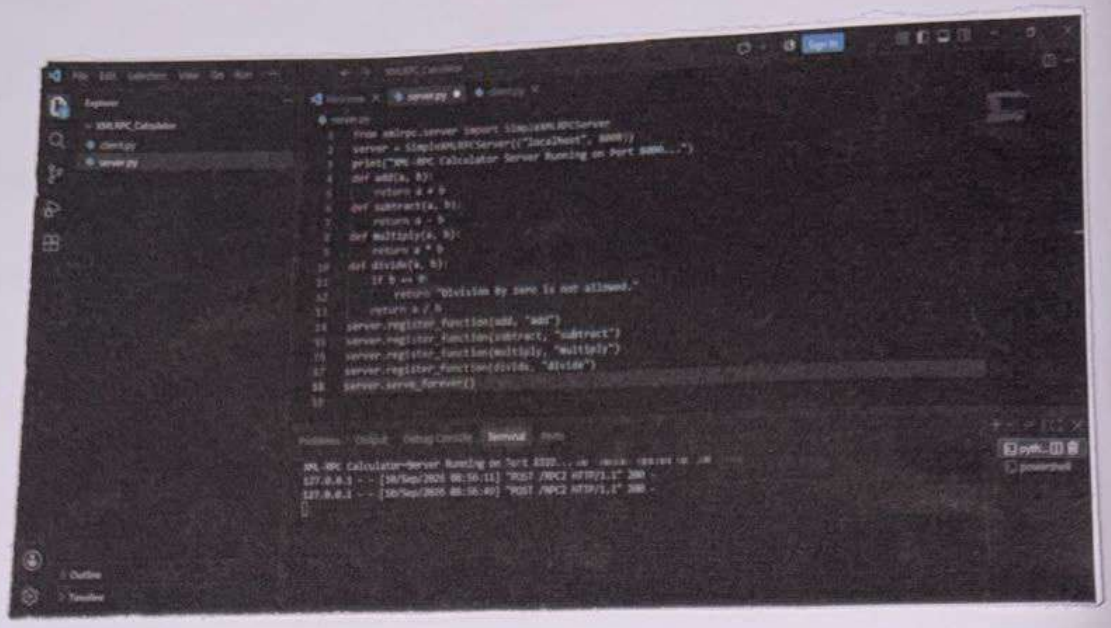
If successfully the server puntis anonymous FTP access.

PROCEDURE:

- Import FTP and all-errors from ftplib.
- Get the FTP server address from the user.
- Connect to the server using port 21
- Display the server welcome message
- Attempt anonymous login
- Close the FTP connection.

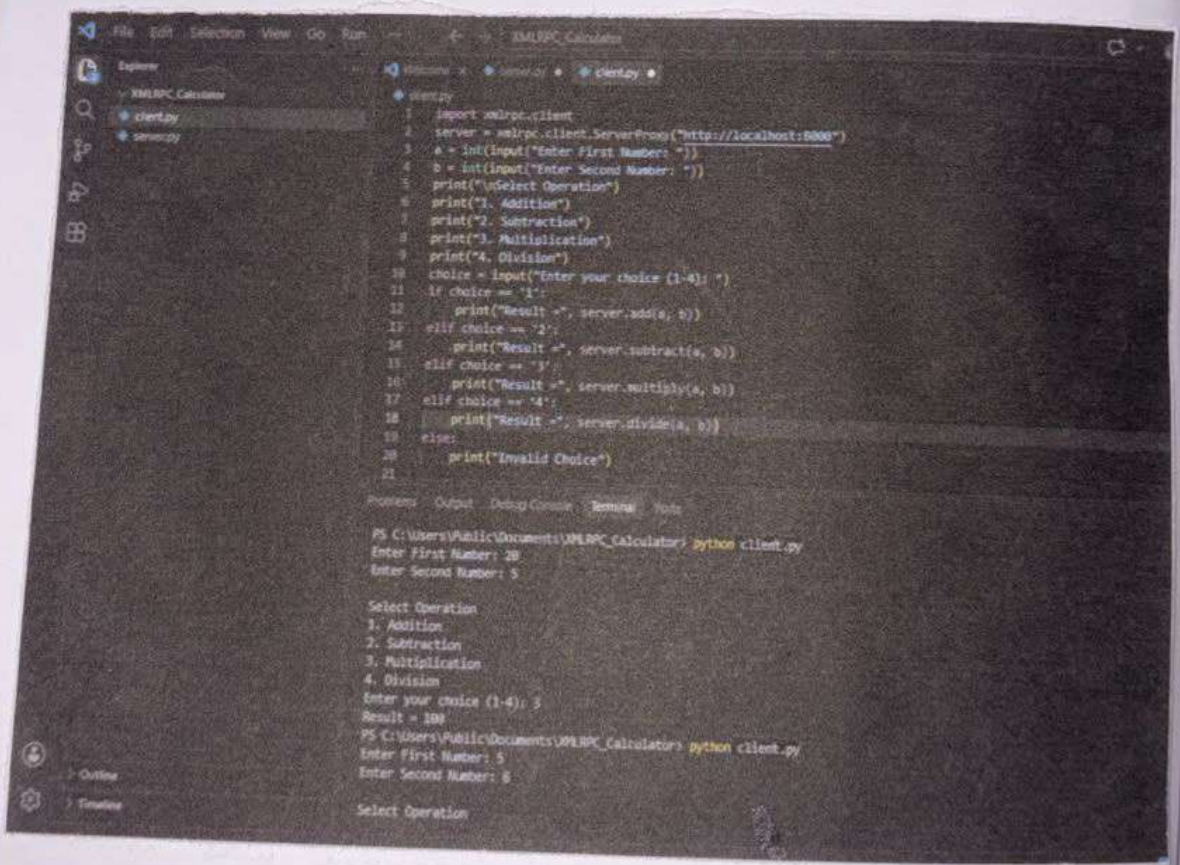
RESULT:

The Anonymous FTP Scanner was successfully implemented. It define where. The authorized FTP server allows anonymous access and display the accessiable files.



```
from xmlrpc.server import SimpleXMLRPCServer
server = SimpleXMLRPCServer(("localhost", 8000))
print("XML-RPC Calculator Server Running on Port 8000...")
def add(a, b):
    return a + b
def subtract(a, b):
    return a - b
def multiply(a, b):
    return a * b
def divide(a, b):
    if b == 0:
        return "Division by zero is not allowed."
    return a / b
server.register_function(add, "add")
server.register_function(subtract, "subtract")
server.register_function(multiply, "multiply")
server.register_function(divide, "divide")
server.serve_forever()
```

```
XML-RPC Calculator-Server Running on Port 8000... (Ctrl+C to exit)
127.0.0.1 - - [16/Sep/2025 08:56:11] "POST /RPC2 HTTP/1.1" 200 -
127.0.0.1 - - [16/Sep/2025 08:56:40] "POST /RPC2 HTTP/1.1" 200 -
```



```
import xmlrpc.client
server = xmlrpc.client.ServerProxy("http://localhost:8000")
a = int(input("Enter First Number: "))
b = int(input("Enter Second Number: "))
print("\nSelect Operation")
print("1. Addition")
print("2. Subtraction")
print("3. Multiplication")
print("4. Division")
choice = input("Enter your choice (1-4): ")
if choice == '1':
    print("Result =", server.add(a, b))
elif choice == '2':
    print("Result =", server.subtract(a, b))
elif choice == '3':
    print("Result =", server.multiply(a, b))
elif choice == '4':
    print("Result =", server.divide(a, b))
else:
    print("Invalid Choice")
```

```
PS C:\Users\Public\Documents\XMLRPC_Calculator> python client.py
Enter First Number: 20
Enter Second Number: 5

Select Operation
1. Addition
2. Subtraction
3. Multiplication
4. Division
Enter your choice (1-4): 3
Result = 100
PS C:\Users\Public\Documents\XMLRPC_Calculator> python client.py
Enter First Number: 5
Enter Second Number: 6

Select Operation
```

Exp: 7

DEVLOP A SIMPLE CALCULATION USING XML-RDC

AIM:

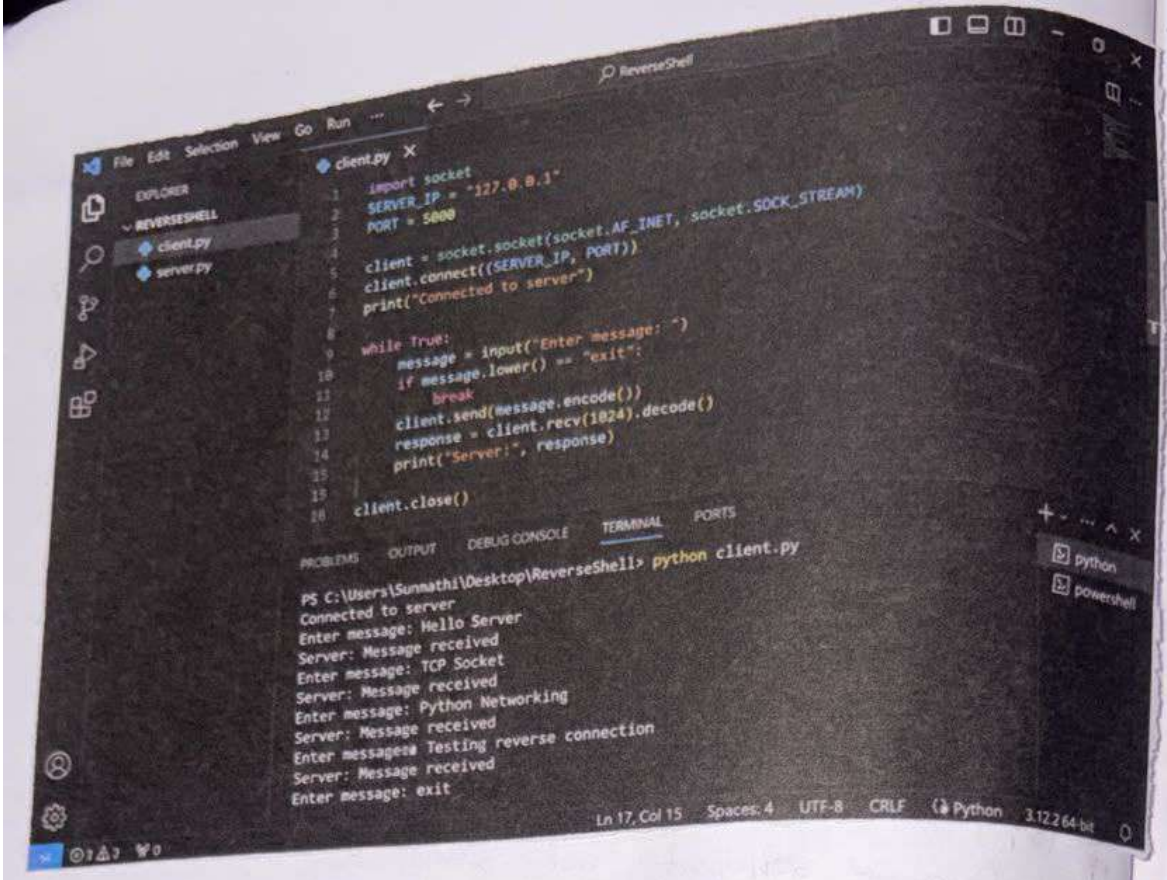
To develop a simple calculator using XML-RDC protocol in python to perform basic arthimetic operation between a client and server.

PROCEDURE:

1. Verify python Installation
2. Create project folder, XMLRDC - calculator w/2 folder inside server.py & client.py.
3. write server.py & client.py programs have the inside the created folder.
4. Open a different and prompt & run earls folder separetly.
5. Start XML RDC server, then server.py & client.py in each cmd prompt.
6. client.py >> Enter values & device prompt

RESULT:

Hence the XMLRDC calculator has been successfully implemented & verified.


ReverseShell
File Edit Selection View Go Run
EXPLORER
REVERSESHELL
client.py
server.py
client.py
import socket
SERVER_IP = "127.0.0.1"
PORT = 5000
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((SERVER_IP, PORT))
print("Connected to server")
while True:
message = input("Enter message: ")
if message.lower() == "exit":
break
client.send(message.encode())
response = client.recv(1024).decode()
print("Server:", response)
client.close()
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Sunmathi\Desktop\ReverseShell> python client.py
Connected to server
Enter message: Hello Server
Server: Message received
Enter message: TCP Socket
Server: Message received
Enter message: Python Networking
Server: Message received
Enter message: Testing reverse connection
Server: Message received
Enter message: exit
python
powershell
Ln 17, Col 15 Spaces: 4 UTF-8 CRLF Python 3.12.2 64-bit


ReverseShell
File Edit Selection View Go Run
EXPLORER
REVERSESHELL
client.py
server.py
server.py
import socket
HOST = "0.0.0.0"
PORT = 5000
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen(1)
print("Waiting for connection...")
conn, addr = server.accept()
print(f"Connected from: {addr}")
while True:
message = conn.recv(1024).decode()
if not message:
break
print(f"Client: {message}")
reply = "Message received"
conn.send(reply.encode())
conn.close()
server.close()
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Sunmathi\Desktop\ReverseShell> python server.py
Waiting for connection...
Connected from: ('127.0.0.1', 54321)
Client: Hello Server
Client: TCP Socket
Client: Python Networking
Client: Testing reverse connection
python
powershell
Ln 18, Col 14 Spaces: 4 UTF-8 CRLF Python 3.12.2 64-bit

Exp: 8

DEVELOP A PROGRAM TO CREAT REVERSE SHELL USING TCP SOCKET

AIM:

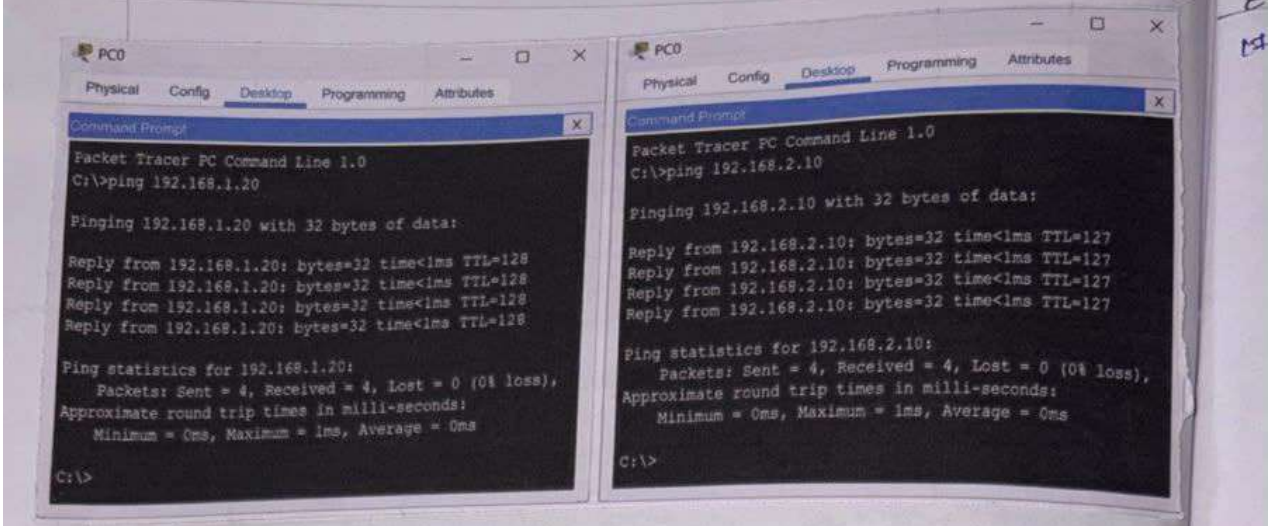
To develop a TCP socket-based client-server program that establishes a connection b/w the client & server enables message communication

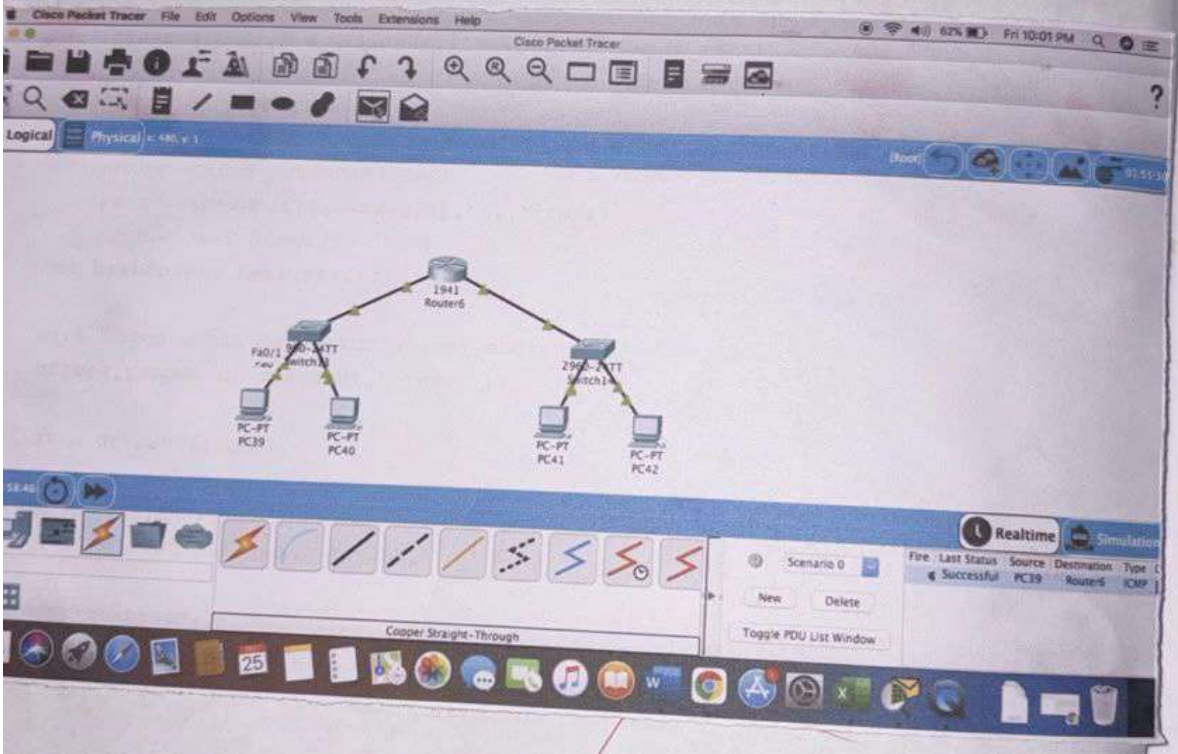
PROCEDURE:

1. Verify the python Installation
2. Create a python file as server.py & client.py
3. Write the python program in server.py & client.py
4. Start the TCP server and bind it to the specified IP
5. Make the client connect to the server using the same IP address & port.
6. Display the recieved response on the client.

RESULT:

Thus, the TCP socket client-server communication program was succesfully implemented in python.


PC0
Physical
Config
Desktop
Programming
Attributes
Command Prompt
Packet Tracer PC Command Line 1.0
C:\>ping 192.168.1.20
Pinging 192.168.1.20 with 32 bytes of data:
Reply from 192.168.1.20: bytes=32 time<1ms TTL=128
Reply from 192.168.1.20: bytes=32 time<1ms TTL=128
Reply from 192.168.1.20: bytes=32 time<1ms TTL=128
Reply from 192.168.1.20: bytes=32 time<1ms TTL=128
Ping statistics for 192.168.1.20:
Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
Minimum = 0ms, Maximum = 1ms, Average = 0ms
C:\>
PC0
Physical
Config
Desktop
Programming
Attributes
Command Prompt
Packet Tracer PC Command Line 1.0
C:\>ping 192.168.2.10
Pinging 192.168.2.10 with 32 bytes of data:
Reply from 192.168.2.10: bytes=32 time<1ms TTL=127
Reply from 192.168.2.10: bytes=32 time<1ms TTL=127
Reply from 192.168.2.10: bytes=32 time<1ms TTL=127
Reply from 192.168.2.10: bytes=32 time<1ms TTL=127
Ping statistics for 192.168.2.10:
Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
Minimum = 0ms, Maximum = 1ms, Average = 0ms
C:\>


Cisco Packet Tracer
File
Edit
Options
View
Tools
Extensions
Help
Logical
Physical
1941
Router6
PC-PT
PC39
PC-PT
PC40
PC-PT
PC41
PC-PT
PC42
Realtime
Simulation
Scenario 0
New
Delete
Toggle PDU List Window
Copper Straight-Through

Exp. 9

1st) 9/26

DESIGN A SIMPLE TOPOLOGY AND CONFIGURE ONE ROUTER, TWO SWITCHES, AND PCs USING CISCO PACKET TRACER

AIM:

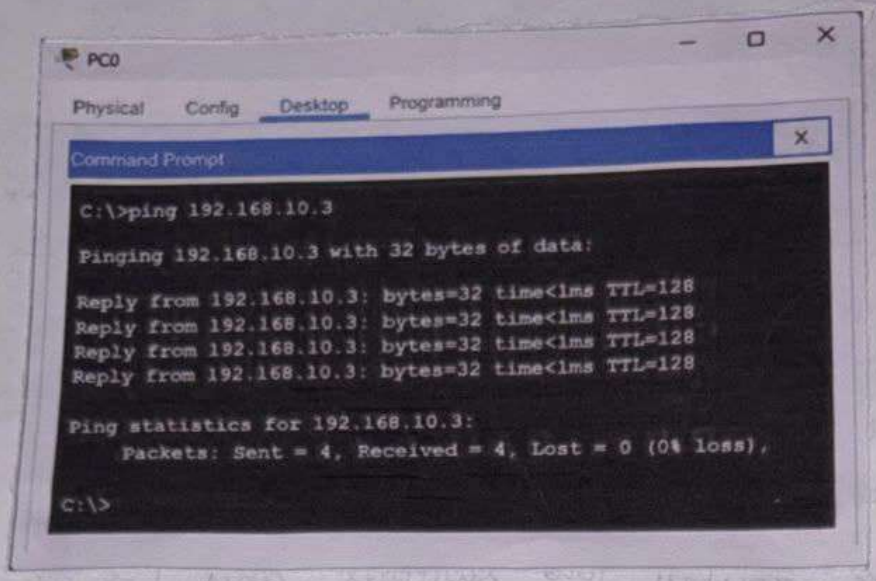
To design and configure a simple network topology using one router, two switches, and PCs in Cisco Packet Tracer.

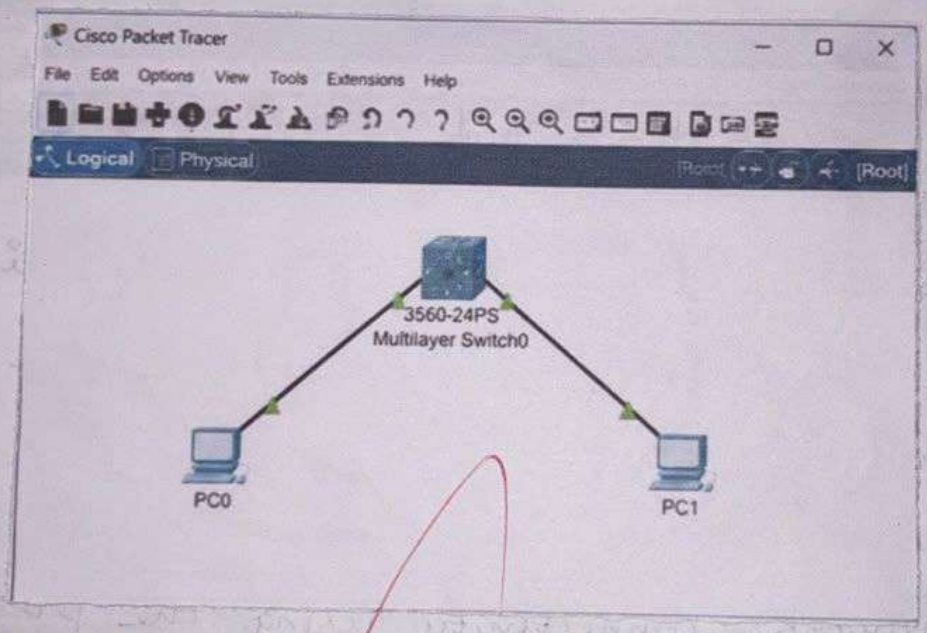
ALGORITHM:

1. Open Cisco Packet Tracer and create a new workspace.
2. Place one router, two switches, and PCs in the workspace
3. Connect the router to both switches using appropriate cables.
4. Connect the PCs to the respective switches
5. Configure IP addresses and subnet masks for all PCs.
6. Configure IP addresses for the router interfaces.
7. Enable the router interfaces using the no shutdown command.
8. Set the default gateway for each PC.
9. Test network connectivity using the ping command.
10. Verify successful communication between the devices.

RESULT:

Thus, a simple network topology using one router, two switches, and PCs was designed and configured successfully in Cisco Packet Tracer. Network connectivity was tested using the ping command.


PC0
Physical
Config
Desktop
Programming
Command Prompt
C:\>ping 192.168.10.3
Pinging 192.168.10.3 with 32 bytes of data:
Reply from 192.168.10.3: bytes=32 time<1ms TTL=128
Reply from 192.168.10.3: bytes=32 time<1ms TTL=128
Reply from 192.168.10.3: bytes=32 time<1ms TTL=128
Reply from 192.168.10.3: bytes=32 time<1ms TTL=128
Ping statistics for 192.168.10.3:
Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
C:\>


Cisco Packet Tracer
File Edit Options View Tools Extensions Help
Logical
Physical
[Root]
3560-24PS
Multilayer Switch0
PC0
PC1

Exp·10
17/9/24

CUSTOMIZE SWITCH WITH NETWORK MODULES USING CISCO PACKET TRACER

AIM:

To customize a switch by adding network modules using Cisco Packet Tracer.

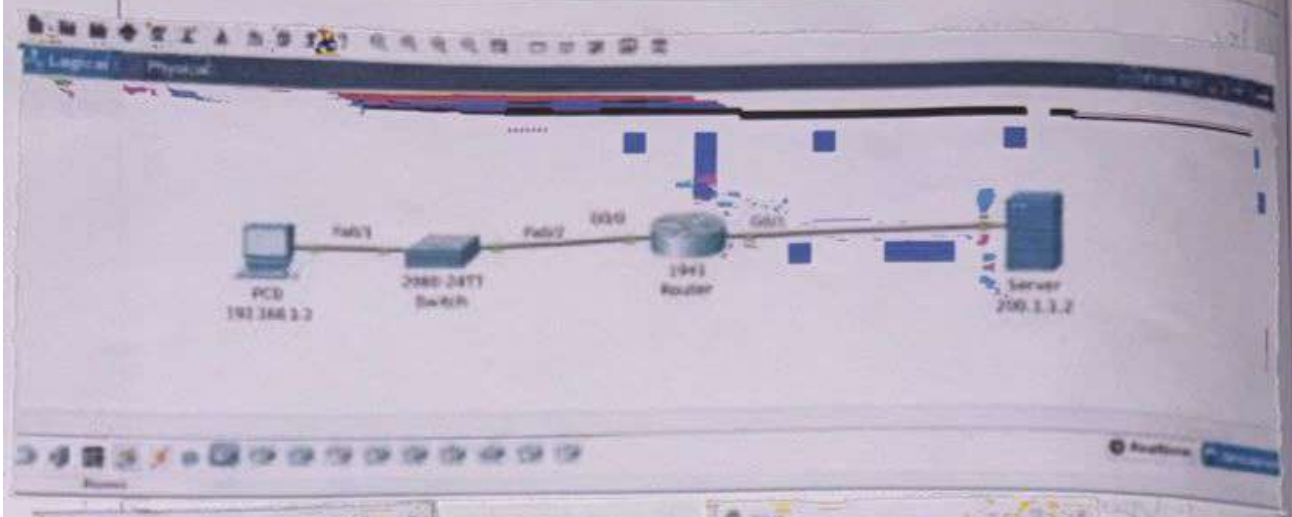
ALGORITHM:

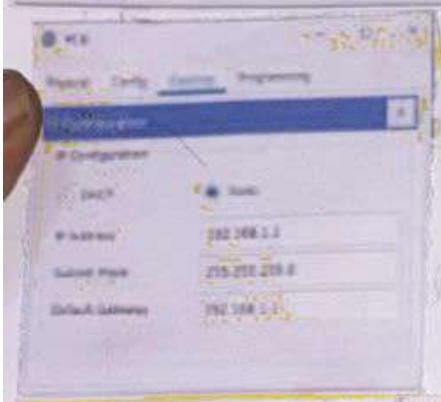
1. Open Cisco Packet Tracer and create a new workspace.
2. Place a switch in the workspace.
3. Select the switch and open the physical tab.
4. Turn off the switch power.
5. Select a suitable network module from the apaile modules
6. Insert the module into the appropriate slot.
7. Turn on the switch power.
8. Connect the switch to other network devices
9. Verify the connections and check the module functionality.

RESULT:

Thus, the switch was customized successfully by adding network modules using Cisco Packet Tracer.

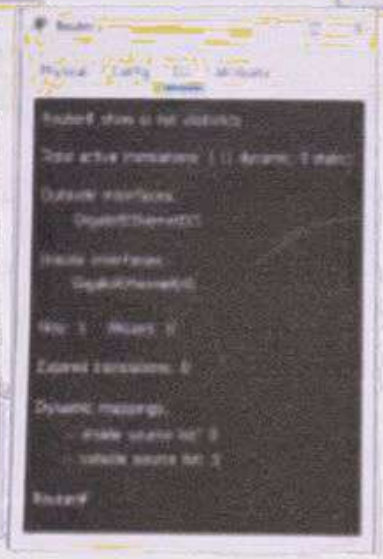
Exp. 11
19/9/26

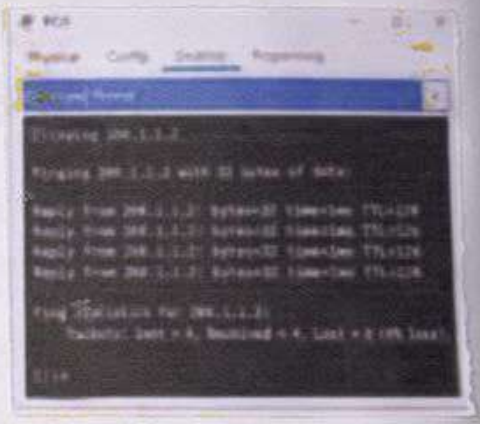

Logical
Physical
Fa0/1
Fa0/2
G0/0
G0/1
PC0
192.168.1.2
2960-24TT
Switch
1941
Router
Server
200.1.1.2

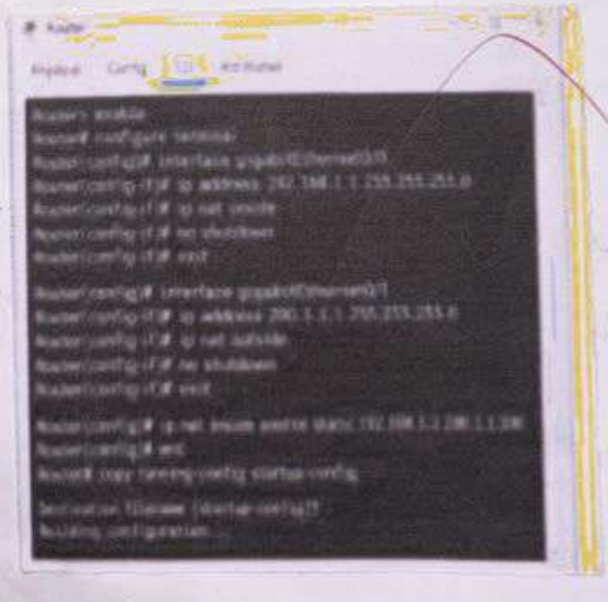

PC0
Physical
Config
Desktop
Programming
IP Configuration
DHCP
Static
IP Address
192.168.1.2
Subnet Mask
255.255.255.0
Default Gateway
192.168.1.1


Physical
Config
Desktop
Programming
IP Configuration
DHCP
Static
IP Address
200.1.1.2
Subnet Mask
255.255.255.0
Default Gateway
200.1.1.1


Router
Physical
Config
CLI
Attributes
Router# show ip nat translations
Pro Inside global Inside local Outside local Outside global
Router#


Router
Physical
Config
CLI
Attributes
Router# show ip nat statistics
Total active translations: 1 (0 dynamic; 1 static)
Outside interfaces:
GigabitEthernet0/0/1
Inside interfaces:
GigabitEthernet0/0/0
Hits: 1 Misses: 0
Expired translations: 0
Dynamic mappings:
-- Inside source list 1
-- Outside source list 1
Router#


PC0
Physical
Config
Desktop
Programming
Pinging 200.1.1.2
Pinging 200.1.1.2 with 32 bytes of data:
Reply from 200.1.1.2: bytes=32 time=1ms TTL=126
Reply from 200.1.1.2: bytes=32 time=1ms TTL=126
Reply from 200.1.1.2: bytes=32 time=1ms TTL=126
Reply from 200.1.1.2: bytes=32 time=1ms TTL=126
Ping statistics for 200.1.1.2:
Packets: Sent = 4, Received = 4, Lost = 0 (0% loss)


Router
Physical
Config
CLI
Attributes
Router> enable
Router# configure terminal
Router(config)# interface gigabitEthernet0/0
Router(config-if)# ip address 192.168.1.1 255.255.255.0
Router(config-if)# ip nat inside
Router(config-if)# no shutdown
Router(config-if)# exit
Router(config)# interface gigabitEthernet0/1
Router(config-if)# ip address 200.1.1.1 255.255.255.0
Router(config-if)# ip nat outside
Router(config-if)# no shutdown
Router(config-if)# exit
Router(config)# ip nat inside source static 192.168.1.2 200.1.1.10
Router(config)# exit
Router# copy running-config startup-config
Destination filename [startup-config]?
Building configuration...

Exp. 11
19/9/26

EXAMINE NETWORK ADDRESS TRANSLATION (NAT) USING CISCO PACKET TRACER

AIM:

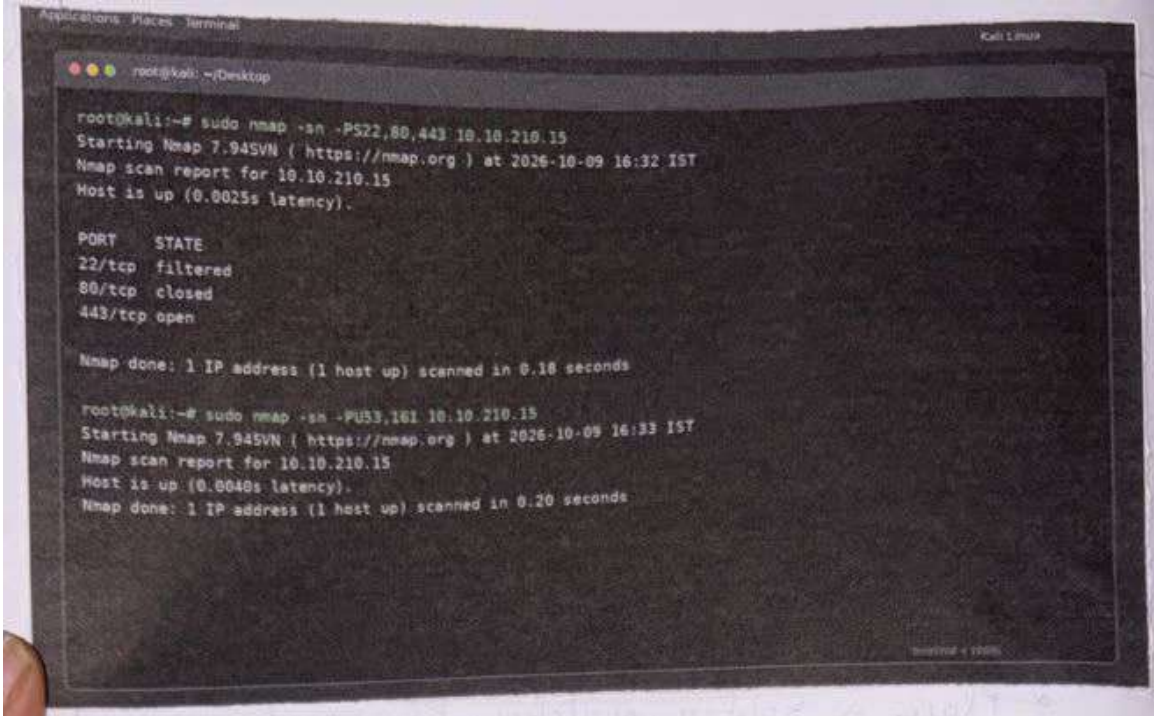
To configure and examine Network Address Translation (NAT) using Cisco Packet Tracer.

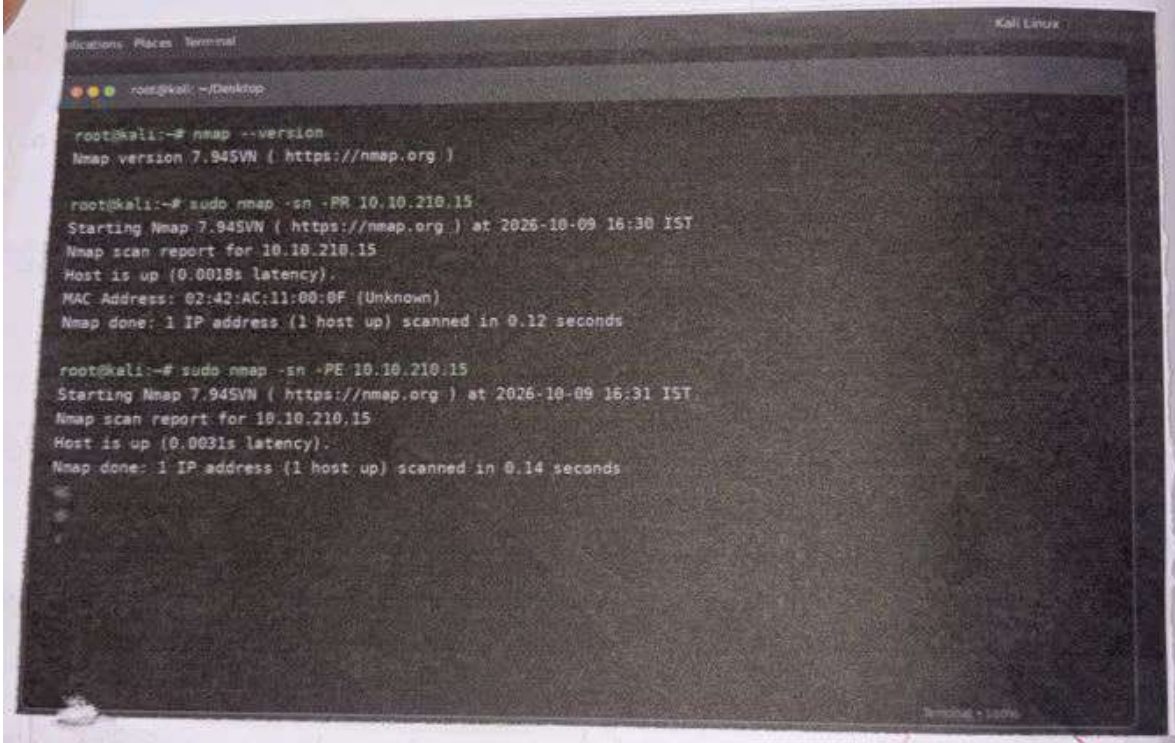
ALGORITHM:

1. Open Cisco Packet Tracer and Create a new workspace.
2. Place a router, switches, PCs, and a server in the workspace.
3. Connect the devices using appropriate cables
4. Configure IP addresses and default gateways for the devices.
5. Configure the router interfaces with suitable IP addresses.
6. Identify the inside and outside NAT interfaces.
7. Configure the NAT translation rules on the router.
8. Configure the appropriate router settings.
9. Generate traffic from the internal network to the external network.
10. Verify NAT translations using the appropriate router commands.

RESULT:

Thus, Network Address Translation (NAT) was configured and examined successfully using Cisco Packet Tracer.


Applications Places Terminal
Kali Linux
root@kali: ~/Desktop
root@kali:~# sudo nmap -sn -PS22,80,443 10.10.210.15
Starting Nmap 7.94SVN (https://nmap.org) at 2026-10-09 16:32 IST
Nmap scan report for 10.10.210.15
Host is up (0.0025s latency).
PORT STATE
22/tcp filtered
80/tcp closed
443/tcp open
Nmap done: 1 IP address (1 host up) scanned in 0.18 seconds
root@kali:~# sudo nmap -sn -PU53,161 10.10.210.15
Starting Nmap 7.94SVN (https://nmap.org) at 2026-10-09 16:33 IST
Nmap scan report for 10.10.210.15
Host is up (0.0040s latency).
Nmap done: 1 IP address (1 host up) scanned in 0.20 seconds


Kali Linux
root@kali: ~/Desktop
root@kali:~# nmap --version
Nmap version 7.94SVN (https://nmap.org)
root@kali:~# sudo nmap -sn -PR 10.10.210.15
Starting Nmap 7.94SVN (https://nmap.org) at 2026-10-09 16:30 IST
Nmap scan report for 10.10.210.15
Host is up (0.0018s latency).
MAC Address: 02:42:AC:11:00:0F (Unknown)
Nmap done: 1 IP address (1 host up) scanned in 0.12 seconds
root@kali:~# sudo nmap -sn -PE 10.10.210.15
Starting Nmap 7.94SVN (https://nmap.org) at 2026-10-09 16:31 IST
Nmap scan report for 10.10.210.15
Host is up (0.0031s latency).
Nmap done: 1 IP address (1 host up) scanned in 0.14 seconds

Exp. 12.
24/9/26

DISCOVERY LIVE HOSTS USING NMAP ON THE TRY HACKME PLATFORM.

AIM:

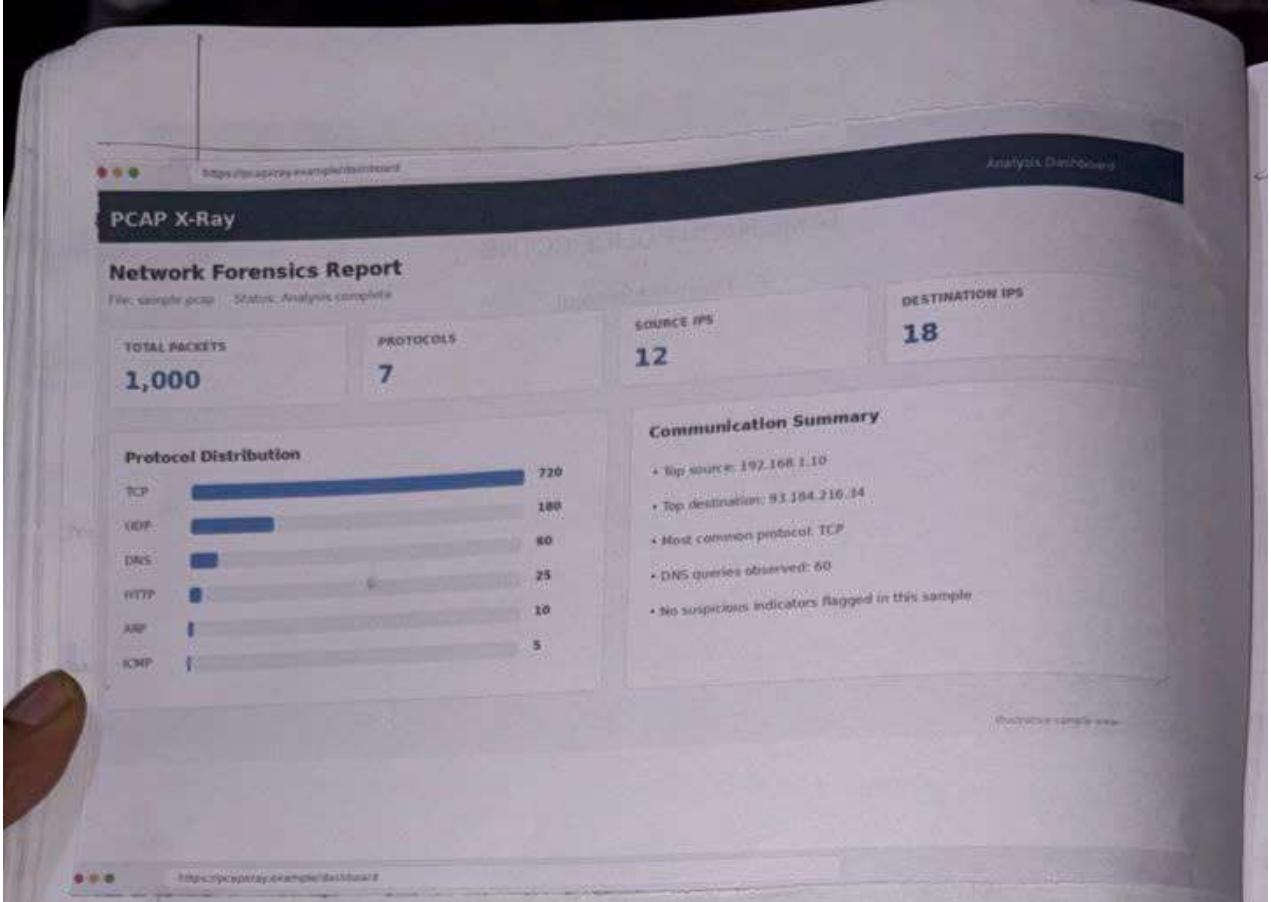
To discover live hosts on an authorized target network using Nmap through ARP scan, ICMP scan, TCP uPing scan, and UDP Ping scan on the TryHackMe Platform

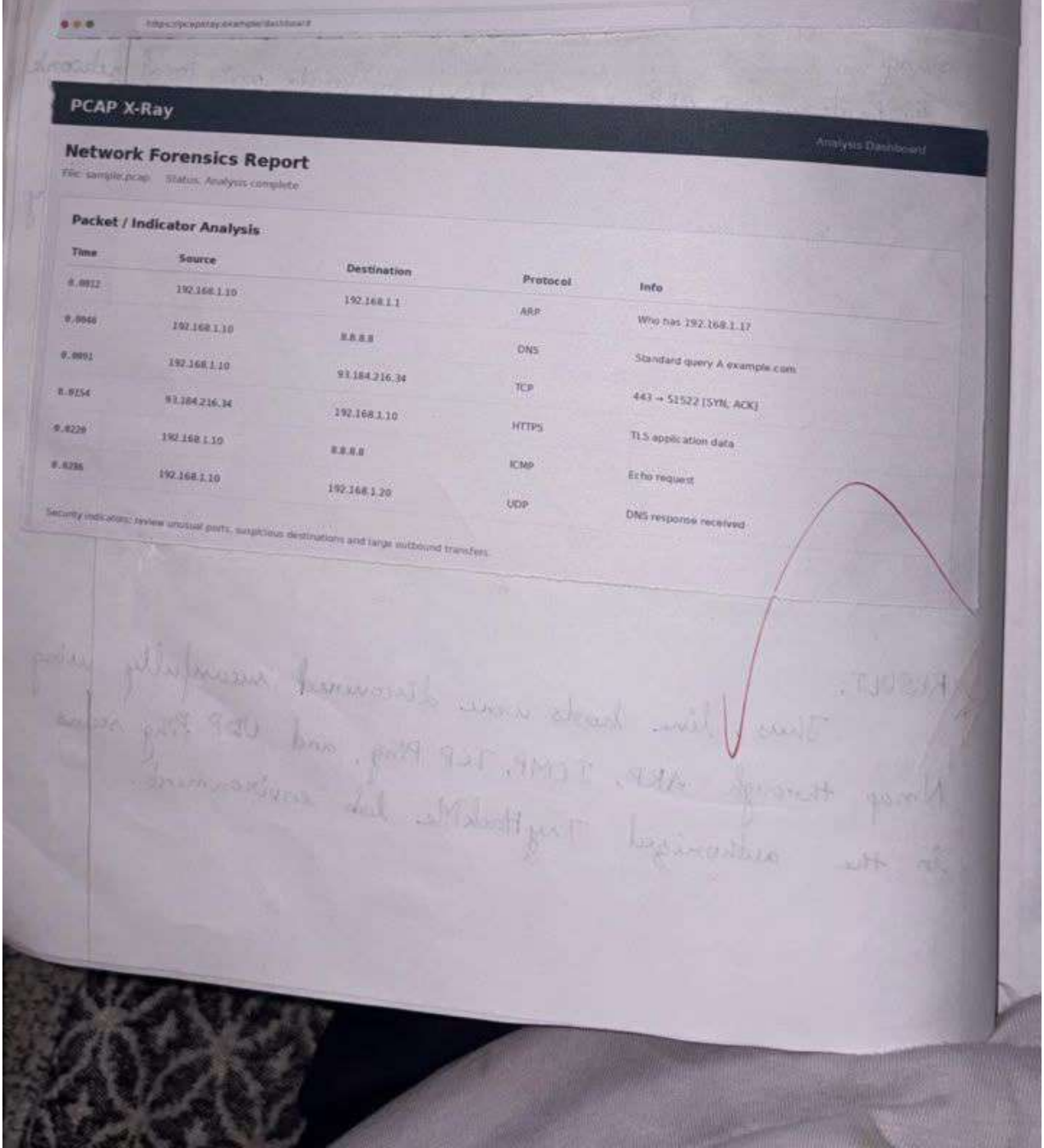
ALGORITHM:

1. Log in to the TryHackMe platform and start the assigned lab environment.
2. Open the terminal in the provided environment.
3. Identify the authorized target IP address or network range.
4. Perform an ARP scan to discover hosts on local network
5. Perform an ICMP scan to identify responsive hosts.
6. Perform a TCP Ping scan to check host availability using TCP probes.
7. Perform a UDP Ping scan to identify hosts responding to UDP probes
8. Observe and record the scan results.
9. Compare the results of the different host discovery techniques.

RESULT:

Thus, live hosts were discovered succesfully using Nmap through ARP, ICMP, TCP Ping, and UDP Ping scans in the authorized TryHackMe lab environment.


https://pcapxray.example/dashboard
Analysis Dashboard
PCAP X-Ray
Network Forensics Report
File: sample.pcap Status: Analysis complete
TOTAL PACKETS
1,000
PROTOCOLS
7
SOURCE IPS
12
DESTINATION IPS
18
Protocol Distribution
TCP 720
UDP 180
DNS 60
HTTP 25
ARP 10
ICMP 5
Communication Summary
• Top source: 192.168.1.10
• Top destination: 93.184.216.34
• Most common protocol: TCP
• DNS queries observed: 60
• No suspicious indicators flagged in this sample


https://pcapxray.example/dashboard
PCAP X-Ray
Analysis Dashboard
Network Forensics Report
File: sample.pcap Status: Analysis complete
Packet / Indicator Analysis
Time Source Destination Protocol Info
0.0012 192.168.1.10 192.168.1.1 ARP Who has 192.168.1.1?
0.0046 192.168.1.10 8.8.8.8 DNS Standard query A example.com
0.0091 192.168.1.10 93.184.216.34 TCP 443 → 51522 [SYN, ACK]
0.0154 93.184.216.34 192.168.1.10 HTTPS TLS application data
0.0220 192.168.1.10 8.8.8.8 ICMP Echo request
0.0286 192.168.1.10 192.168.1.20 UDP DNS response received
Security indicators: review unusual ports, suspicious destinations and large outbound transfers.

Exp 13
1/10/24

DEMONSTRATE NETWORK FORENSICS USING PCAP X-RAY TOOL

AIM:

To demonstrate network forensics by analyzing PCAP files using the PCAP X-ray tool.

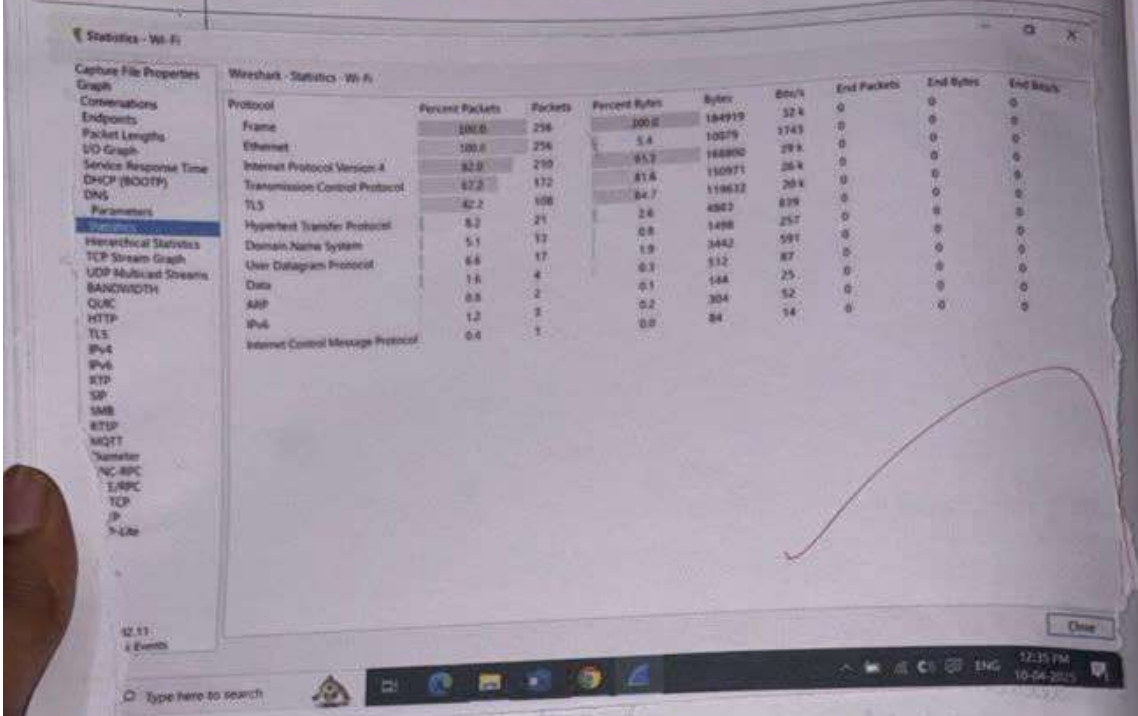
ALGORITHM:

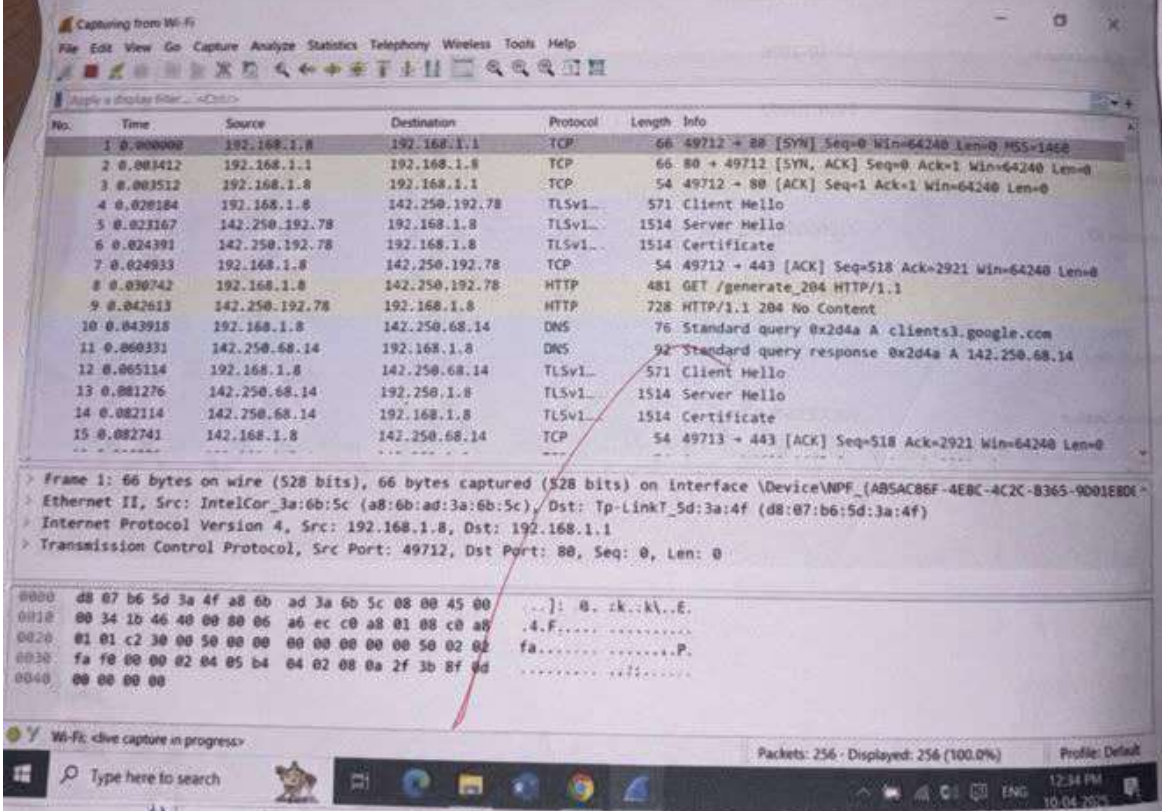
1. Open the PCAP X-ray tool.
2. Upload the captured PCAP file.
3. Start the analysis process.
4. Examine the network packets and communication details.
5. Identify source and distination IP addresses.
6. Analyze Protocols and suspicious network activities
7. Observe and record the analysis results.

RESULT:

Thus, network forensics was demonstrated successfully by analyzing PCAP files using the PCAP X-ray tool.

Exp. 14
7/10/26


Statistics - Wi-Fi
Wireshark · Statistics · Wi-Fi
Capture File Properties
Graph
Conversations
Endpoints
Packet Lengths
I/O Graph
Service Response Time
DHCP (BOOTP)
DNS
Parameters
Protocol Hierarchy
Hierarchical Statistics
TCP Stream Graph
UDP Multicast Streams
BANDWIDTH
QUIC
HTTP
TLS
IPv4
IPv6
RTP
SIP
SMB
RTSP
MQTT
Close
Type here to search
ENG


Capturing from Wi-Fi
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
No. Time Source Destination Protocol Length Info
1 0.000000 192.168.1.8 192.168.1.1 TCP 66 49712 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460
2 0.003412 192.168.1.1 192.168.1.8 TCP 66 80 → 49712 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0
3 0.003512 192.168.1.8 192.168.1.1 TCP 54 49712 → 80 [ACK] Seq=1 Ack=1 Win=64240 Len=0
4 0.020184 192.168.1.8 142.250.192.78 TLSv1... 571 Client Hello
5 0.023167 142.250.192.78 192.168.1.8 TLSv1... 1514 Server Hello
6 0.024391 142.250.192.78 192.168.1.8 TLSv1... 1514 Certificate
7 0.024933 192.168.1.8 142.250.192.78 TCP 54 49712 → 443 [ACK] Seq=518 Ack=2921 Win=64240 Len=0
8 0.030742 192.168.1.8 142.250.192.78 HTTP 481 GET /generate_204 HTTP/1.1
9 0.042613 142.250.192.78 192.168.1.8 HTTP 728 HTTP/1.1 204 No Content
10 0.043918 192.168.1.8 142.250.68.14 DNS 76 Standard query 0x2d4a A clients3.google.com
11 0.060331 142.250.68.14 192.168.1.8 DNS 92 Standard query response 0x2d4a A 142.250.68.14
12 0.065114 192.168.1.8 142.250.68.14 TLSv1... 571 Client Hello
13 0.081276 142.250.68.14 192.250.1.8 TLSv1... 1514 Server Hello
14 0.082114 142.250.68.14 192.168.1.8 TLSv1... 1514 Certificate
15 0.082741 142.168.1.8 142.250.68.14 TCP 54 49713 → 443 [ACK] Seq=518 Ack=2921 Win=64240 Len=0
Frame 1: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface \Device\NPF_{AB5AC86F-4E8C-4C2C-B365-9D01E8D6
Ethernet II, Src: IntelCor_3a:6b:5c (a8:6b:ad:3a:6b:5c), Dst: Tp-LinkT_5d:3a:4f (d8:07:b6:5d:3a:4f)
Internet Protocol Version 4, Src: 192.168.1.8, Dst: 192.168.1.1
Transmission Control Protocol, Src Port: 49712, Dst Port: 80, Seq: 0, Len: 0
0000 d8 07 b6 5d 3a 4f a8 6b ad 3a 6b 5c 08 00 45 00
0010 00 34 1b 46 40 00 80 06 a6 ec c0 a8 01 08 c0 a8
0020 01 01 c2 30 00 50 00 00 00 00 00 00 00 00 50 02
0030 fa f0 00 00 02 04 05 b4 04 02 08 0a 2f 3b 8f 0d
0040 00 00 00 00
Wi-Fi: <live capture in progress>
Packets: 256 · Displayed: 256 (100.0%)
Profile: Default
Type here to search
ENG

Exp. 14
7/10/26

To Capture, Save, and Analyze Network Traffic using wireshark Tool

Aim:

To capture, save, and analyze network traffic on TCP, UDP, IP, HTTP, APP, DHCP, ICMP, and DNS using wireshark.

ALGORITHM:

1. open wireshark and select a network interface.
2. Start capturing network traffic.
3. Generate traffic using different network Protocols.
4. Apply filters to identify the required Protocols.
5. Analyze Packet details and IP addresses.
6. Stop the capture and save the Packets
7. Examine the captured traffic and record the observations.

RESULT:

Thus, network traffic was captured, Seved, and analyzed successfully using wireshark.

http://localhost/usage/

Webalizer Web Server Statistics

Usage Statistics for localhost

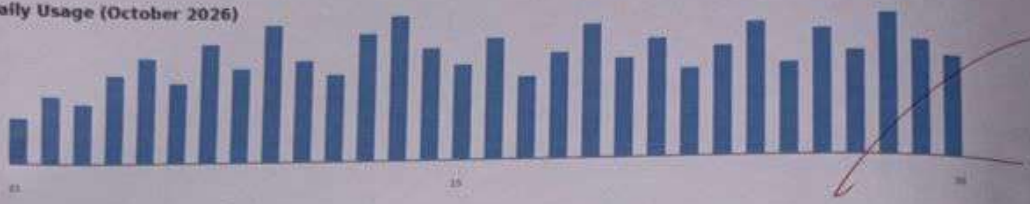
Summary period: October 2026 | Log source: access.log

Hits	Files	Pages	Visits	Sites
1,248	986	412	173	48

Monthly History

Month	Hits	Files	Pages	Visits	Sites
Oct 2026	1,248	986	412	173	48
Sep 2026	1,106	874	388	151	42
Aug 2026	982	771	340	139	39

Daily Usage (October 2026)



Webalizer Web Server Statistics

Usage Statistics for localhost

Summary period: October 2026 | Log source: access.log

Hits	Files	Pages	Visits	Sites
1,248	986	412	173	48

Daily Statistics and Request Summary

Top Requested Pages

/	245
/index.html	184
/about.html	121
/contact.html	96
/images/logo.png	83
/products.html	71

HTTP Response Codes

200 OK	870
304 Not Modified	190
404 Not Found	132
301 Redirect	56

Exp: 15
12/10/26

To Analyze Different Types of Servers Using webalizer Tool

AIM:

To analyze web server log files using the webalizer tool and generate traffic statistics

ALGORITHM:

1. Install and open the webalizer tool.
2. Locate the web server log file.
3. Configure the log file path.
4. Run webalizer to process the log data.
5. Generate traffic reports.
6. Analyze visits, Page views, requests, and bandwidth usage.
7. Record the observations.

RESULT:

Thus, web server log files were analyzed successfully using webalizer, and traffic stats were generated.